

МОСКОВСКИЙ АВИАЦИОННЫЙ ИНСТИТУТ  
(НАЦИОНАЛЬНЫЙ ИССЛЕДОВАТЕЛЬСКИЙ УНИВЕРСИТЕТ)

Институт №8 «Компьютерные науки и прикладная математика»  
Кафедра 806 «Вычислительная математика и программирование»

**Лабораторная работа №1**  
**по курсу «Операционные системы»**

Выполнил: А. В. Козлов  
Группа: М8О-207БВ-24  
Преподаватель: Е. С. Миронов

Москва, 2025

## Условие

**Цель работы:** Приобретение практических навыков в:

- Управление процессами в ОС
- Обеспечение обмена данных между процессами посредством каналов

**Задание:** Составить и отладить программу на языке Си, осуществляющую работу с процессами и взаимодействие между ними в одной из двух операционных систем. В результате работы программа (основной процесс) должен создать для решение задачи один или несколько дочерних процессов. Взаимодействие между процессами осуществляется через системные сигналы/события и/или каналы (pipe). Необходимо обрабатывать системные ошибки, которые могут возникнуть в результате работы.

Родительский процесс создает дочерний процесс. Первой строкой пользователь в консоль родительского процесса вводит имя файла, которое будет использовано для открытия File с таким именем на запись. Перенаправление стандартных потоков ввода-вывода показано на картинке выше. Родительский и дочерний процесс должны быть представлены разными программами. Родительский процесс принимает от пользователя строки произвольной длины и пересылает их в pipe1. Процесс child проверяет строки на валидность правилу. Если строка соответствует правилу, то она выводится в стандартный поток вывода дочернего процесса, иначе в pipe2 выводится информация об ошибке. Родительский процесс полученные от child ошибки выводит в стандартный поток вывода.

**Вариант: 15**

## Метод решения

Родительский процесс принимает пользовательский ввод имени файла для записи валидных строк, создаёт или открывает его. Далее создаются два канала (pipe1, pipe2) и один дочерний процесс (child). Родитель читает строки из стандартного ввода и отправляет их в первый канал. Дочерний процесс получает строки из канала, проверяет их на валидность (начинается ли строка с заглавной буквы) и записывает валидные строки в файл, а информацию о невалидных строках отправляет обратно родителю через второй канал. Родительский процесс выводит полученные ошибки в стандартный поток вывода.

## Описание программы

Файл os.h содержит общие интерфейсы для системных вызовов. Реализация этих функций для Linux находится в os\_linux.c.

В parent.c:

Просим пользователя ввести имя файла

Создаем две трубы (pipe1 и pipe2) через CreatePipe

Создаем дочерний процесс через CloneProcess

Читаем строки с клавиатуры и отправляем их в pipe1

Получаем сообщения об ошибках из pipe2 и показываем их на экране

Ждем когда дочерний процесс завершится через WaitProcess

В child.c:

Получаем имя файла из аргументов командной строки

Открываем файл для записи

Читаем строки из STDIN (который подключен к pipe1)

Проверяем каждую строку - должна начинаться с большой буквы

Если строка правильная - пишем в файл

Если строка неправильная - отправляем сообщение об ошибке в STDOUT (который подключен к pipe2)

Основные типы данных:

pipe\_t - дескриптор канала

process\_t - идентификатор процесса

Используемые системные вызовы:

pipe - создание канала

fork - создание дочернего процесса

execv - запуск новой программы

dup2 - перенаправление потоков ввода-вывода

write - запись в канал

read - чтение из канала

wait - ожидание завершения процесса

close - закрытие дескриптора

## Результаты

При запуске программы, вводе имени файла и тестовых строк, валидные строки (начинающиеся с заглавной буквы) записываются в указанный файл, а информация о невалидных строках выводится в консоль.

Пример работы:

Ввод имени файла:

output.txt

Ввод строк:

Hello world

invalid line

Another valid line

test string

Last example

Содержимое output.txt:

Hello world

Another valid line

Last example

Вывод в консоль:

line "invalid line" is not valid

line "test string" is not valid

## **Выводы**

Разработанная программа успешно реализует заданное взаимодействие процессов через каналы pipe. Родительский процесс корректно передает строки дочернему процессу, который выполняет валидацию по правилу (первая буква - заглавная). Валидные строки записываются в файл, информация о невалидных строках возвращается родительскому процессу и выводится в консоль. Межпроцессное взаимодействие организовано через два канала, обеспечивая двунаправленную передачу данных.

## Исходная программа

```
1 | #pragma once
2 |
3 | #ifdef _WIN32
4 |
5 | #else
6 | #include <errno.h>
7 | #include <fcntl.h>
8 | #include <stdio.h>
9 | #include <stdlib.h>
10 | #include <string.h>
11 | #include <sys/wait.h>
12 | #include <unistd.h>
13 |
14 | typedef int pipe_t;
15 | typedef int pid_t;
16 |
17 | typedef struct {
18 |     pid_t pid;
19 | } proc_info_t;
20 | #endif
21 |
22 | typedef int pipe_t;
23 | typedef int pid_t;
24 |
25 | int CreatePipe(pipe_t new_pipe[2]);
26 |
27 | pid_t CloneProcess();
28 |
29 | int Exec(const char* path, char* argv[]);
30 |
31 | int LinkFDtoIN(int fd);
32 |
33 | int LinkFDtoOUT(int fd);
34 |
35 | int OpenObject(const char* path, int flags, int mod);
36 |
37 | int WaitProcess();
38 |
39 | int ClosePipe(pipe_t pipe);
40 |
41 | int WritePipe(pipe_t pipe, void* buffer, int bytes);
42 |
43 | int ReadPipe(pipe_t pipe, void* buffer, int bytes);
```

Листинг 1: \*Интерфейс вызовов\*

```
1 | #include "os.h"
2 |
3 | #include <stdio.h>
4 | #include <stdlib.h>
5 | #include <string.h>
6 | #include <unistd.h>
7 | #include <sys/wait.h>
8 |
9 | int CreatePipe(pipe_t new_pipe[2]) { return pipe(new_pipe); }
```

```

10
11 pid_t CloneProcess() { return fork(); }
12
13 int Exec(const char* path, char* argv[]) {
14     return execv(path, argv);
15 }
16
17 int LinkFDtoIN(int fd) {
18     return dup2(fd, STDIN_FILENO);
19 }
20
21 int LinkFDtoOUT(int fd) {
22     return dup2(fd, STDOUT_FILENO);
23 }
24
25 int OpenObject(const char* path, int flags, int mod) {
26     return open(path, flags, mod);
27 }
28
29 int WaitProcess() { return wait(NULL); }
30
31 int ClosePipe(pipe_t pipe) { return close(pipe); }
32
33 int WritePipe(pipe_t pipe, void* buffer, int bytes) {
34     return write(pipe, buffer, bytes);
35 }
36
37 int ReadPipe(pipe_t pipe, void* buffer, int bytes) {
38     return read(pipe, buffer, bytes);
39 }

```

Листинг 2: \*Реализация вызовов\*

```

1  #include "os.h"
2
3  #include <stdio.h>
4
5  #define BUFFERSIZE 256
6
7  int main() {
8      int pipe1[2];
9      int pipe2[2];
10     pid_t pid;
11     char filename[BUFFERSIZE];
12
13     printf(" ");
14     if(!fgets(filename, BUFFERSIZE, stdin)) {
15         perror("fgets error");
16         exit(1);
17     }
18
19     filename[strlen(filename)] = 0;
20
21     if(CreatePipe(pipe1) == -1 || CreatePipe(pipe2) == -1) {
22         perror("pipe error");
23         exit(1);
24     }

```

```

25
26     pid = CloneProcess();
27     if(pid == -1) {
28         perror("fork error");
29         exit(1);
30     }
31
32     if(pid == 0) {
33         // Or 1w
34         ClosePipe(pipe1[1]);
35         ClosePipe(pipe2[0]);
36         LinkFDtoIN(pipe1[0]);
37         ClosePipe(pipe1[0]);
38         LinkFDtoOUT(pipe2[1]);
39         ClosePipe(pipe2[1]);
40
41         Exec("./child", (char*[]){ "child", filename, NULL });
42         perror("execl error");
43         exit(1);
44     } else {
45         ClosePipe(pipe1[0]);
46         ClosePipe(pipe2[1]);
47         char buffer[BUFFERSIZE];
48         while(fgets(buffer, BUFFERSIZE, stdin)) {
49             WritePipe(pipe1[1], buffer, strlen(buffer));
50         }
51         ClosePipe(pipe1[1]);
52
53         int bytes;
54         while((bytes = read(pipe2[0], buffer, BUFFERSIZE)) > 0) {
55             fwrite(buffer, 1, bytes, stdout);
56         }
57         if(bytes < 0) {
58             perror("can't read from pipe2");
59         }
60         ClosePipe(pipe2[0]);
61         wait(NULL);
62     }
63 }

```

Листинг 3: \*Родительский процесс\*

```

1  #include "os.h"
2
3  #include <stdio.h>
4  #include <ctype.h>
5
6  #define BUFFERSIZE 256
7
8  int main(int argc, char* argv[]) { // argv [0] - child, [1] - filename
9      char buffer[BUFFERSIZE];
10     FILE* file;
11
12     file = fopen(argv[1], "w");
13     if(file == NULL) {
14         perror("file trouble");
15         exit(1);

```

```

16     }
17     while(fgets(buffer, BUFFERSIZE, stdin) != NULL) {
18         if(isupper(buffer[0])) {
19             fprintf(file, "%s", buffer);
20         } else {
21             printf("line %s is not valid\n", buffer);
22         }
23     }
24     fclose(file);
25 }

```

Листинг 4: \*Дочерний процесс\*

```

14159 execve("./parent",["./parent"],0x7ffdd1ac1498 /* 48 vars */)
= 0
14159 brk(NULL) = 0x5dc700896000
14159
mmap(NULL,8192,PROT_READ|PROT_WRITE,MAP_PRIVATE|MAP_ANONYMOUS,-1,0) =
0x7c6fd99c6000
14159 access("/etc/ld.so.preload",R_OK) = -1 ENOENT (No such file
or directory)
14159 openat(AT_FDCWD,"/etc/ld.so.cache",O_RDONLY|O_CLOEXEC) = 3
14159 fstat(3,{st_mode=S_IFREG|0644,st_size=59259,...}) = 0
14159 mmap(NULL,59259,PROT_READ,MAP_PRIVATE,3,0) = 0x7c6fd99b7000
14159 close(3) = 0
14159
openat(AT_FDCWD,"/lib/x86_64-linux-gnu/libc.so.6",O_RDONLY|O_CLOEXEC) = 3
14159
read(3,"\177ELF\2\1\1\3\0\0\0\0\0\0\0\0\3\0>\0\1\0\0\0\220\243\2\0\0\0\0\0"... ,832)
= 832
14159
pread64(3,"\6\0\0\0\4\0\0\0@\0\0\0\0\0\0\0@\0\0\0\0\0\0\0@\0\0\0\0\0\0\0"... ,784,64)
= 784
14159 fstat(3,{st_mode=S_IFREG|0755,st_size=2125328,...}) = 0
14159
pread64(3,"\6\0\0\0\4\0\0\0@\0\0\0\0\0\0\0@\0\0\0\0\0\0\0@\0\0\0\0\0\0\0"... ,784,64)
= 784
14159 mmap(NULL,2170256,PROT_READ,MAP_PRIVATE|MAP_DENYWRITE,3,0) =
0x7c6fd9600000
14159
mmap(0x7c6fd9628000,1605632,PROT_READ|PROT_EXEC,MAP_PRIVATE|MAP_FIXED|MAP_DENYWRITE,3,0)
= 0x7c6fd9628000
14159
mmap(0x7c6fd97b0000,323584,PROT_READ,MAP_PRIVATE|MAP_FIXED|MAP_DENYWRITE,3,0x1b0000)
= 0x7c6fd97b0000
14159
mmap(0x7c6fd97ff000,24576,PROT_READ|PROT_WRITE,MAP_PRIVATE|MAP_FIXED|MAP_DENYWRITE,3,0)
= 0x7c6fd97ff000
14159

```



```

mmap(0x7c6fd9805000,52624,PROT_READ|PROT_WRITE,MAP_PRIVATE|MAP_FIXED|MAP_ANONYMOUS,-1
= 0x7c6fd9805000
    14159 close(3) = 0
    14159
mmap(NULL,12288,PROT_READ|PROT_WRITE,MAP_PRIVATE|MAP_ANONYMOUS,-1,0) =
0x7c6fd99b4000
    14159 arch_prctl(ARCH_SET_FS,0x7c6fd99b4740) = 0
    14159 set_tid_address(0x7c6fd99b4a10) = 14159
    14159 set_robust_list(0x7c6fd99b4a20,24) = 0
    14159 rseq(0x7c6fd99b5060,0x20,0,0x53053053) = 0
    14159 mprotect(0x7c6fd97ff000,16384,PROT_READ) = 0
    14159 mprotect(0x5dc6ec28d000,4096,PROT_READ) = 0
    14159 mprotect(0x7c6fd9a04000,8192,PROT_READ) = 0
    14159
prlimit64(0,RLIMIT_STACK,NULL,{rlim_cur=8192*1024,rlim_max=RLIM64_INFINITY})
= 0
    14159 munmap(0x7c6fd99b7000,59259) = 0
    14159 fstat(1,{st_mode=S_IFCHR|0620,st_rdev=makedev(0x88,0),...}) =
0
    14159 getrandom("\x5e\x83\x1f\xef\xd4\x45\xf7\xf7",8,GRND_NONBLOCK)
= 8
    14159 brk(NULL) = 0x5dc700896000
    14159 brk(0x5dc7008b7000) = 0x5dc7008b7000
    14159 fstat(0,{st_mode=S_IFCHR|0620,st_rdev=makedev(0x88,0),...}) =
0
    14159
write(1,"\320\222\320\262\320\265\320\264\320\270\321\202\320\265
\320\270\320\274\321\217 \321\204\320\260\320\271\320\273\320\260",32) =
32
    14159 read(0,"1.txt\n",1024) = 6
    14159 pipe2([3,4],0) = 0
    14159 pipe2([5,6],0) = 0
    14159
clone(child_stack=NULL,flags=CLONE_CHILD_CLEARTID|CLONE_CHILD_SETTID|SIGCHLD,child_ti
= 14160
    14160 set_robust_list(0x7c6fd99b4a20,24 <unfinished ...>
    14159 close(3) = 0
    14160 <... set_robust_list resumed> = 0
    14159 close(6) = 0
    14160 close(4) = 0
    14160 close(5) = 0
    14159 read(0, <unfinished ...>
    14160 dup2(3,0) = 0
    14160 close(3) = 0
    14160 dup2(6,1) = 1
    14160 close(6) = 0
    14160 execve("./child",["child","1.txt"],0x7ffdb2375ee8 /* 48 vars
*/ = 0

```

```

14160 brk(NULL) = 0x5c955baf9000
14160
mmap(NULL,8192,PROT_READ|PROT_WRITE,MAP_PRIVATE|MAP_ANONYMOUS,-1,0) =
0x718f19ac9000
14160 access("/etc/ld.so.preload",R_OK) = -1 ENOENT (No such file
or directory)
14160 openat(AT_FDCWD,"/etc/ld.so.cache",O_RDONLY|O_CLOEXEC) = 3
14160 fstat(3,{st_mode=S_IFREG|0644,st_size=59259,...}) = 0
14160 mmap(NULL,59259,PROT_READ,MAP_PRIVATE,3,0) = 0x718f19aba000
14160 close(3) = 0
14160
openat(AT_FDCWD,"/lib/x86_64-linux-gnu/libc.so.6",O_RDONLY|O_CLOEXEC) = 3
14160
read(3,"\177ELF\2\1\1\3\0\0\0\0\0\0\0\0\3\0>\0\1\0\0\0\220\243\2\0\0\0\0\0"... ,832)
= 832
14160
pread64(3,"\6\0\0\0\4\0\0\0@\0\0\0\0\0\0\0@\0\0\0\0\0\0\0@\0\0\0\0\0\0\0"... ,784,64)
= 784
14160 fstat(3,{st_mode=S_IFREG|0755,st_size=2125328,...}) = 0
14160
pread64(3,"\6\0\0\0\4\0\0\0@\0\0\0\0\0\0\0@\0\0\0\0\0\0\0@\0\0\0\0\0\0\0"... ,784,64)
= 784
14160 mmap(NULL,2170256,PROT_READ,MAP_PRIVATE|MAP_DENYWRITE,3,0) =
0x718f19800000
14160
mmap(0x718f19828000,1605632,PROT_READ|PROT_EXEC,MAP_PRIVATE|MAP_FIXED|MAP_DENYWRITE,3,0)
= 0x718f19828000
14160
mmap(0x718f199b0000,323584,PROT_READ,MAP_PRIVATE|MAP_FIXED|MAP_DENYWRITE,3,0x1b0000)
= 0x718f199b0000
14160
mmap(0x718f199ff000,24576,PROT_READ|PROT_WRITE,MAP_PRIVATE|MAP_FIXED|MAP_DENYWRITE,3,0)
= 0x718f199ff000
14160
mmap(0x718f19a05000,52624,PROT_READ|PROT_WRITE,MAP_PRIVATE|MAP_FIXED|MAP_ANONYMOUS,-1,0)
= 0x718f19a05000
14160 close(3) = 0
14160
mmap(NULL,12288,PROT_READ|PROT_WRITE,MAP_PRIVATE|MAP_ANONYMOUS,-1,0) =
0x718f19ab7000
14160 arch_prctl(ARCH_SET_FS,0x718f19ab7740) = 0
14160 set_tid_address(0x718f19ab7a10) = 14160
14160 set_robust_list(0x718f19ab7a20,24) = 0
14160 rseq(0x718f19ab8060,0x20,0,0x53053053) = 0
14160 mprotect(0x718f199ff000,16384,PROT_READ) = 0
14160 mprotect(0x5c95540e8000,4096,PROT_READ) = 0
14160 mprotect(0x718f19b07000,8192,PROT_READ) = 0
14160

```

```

prlimit64(0,RLIMIT_STACK,NULL,{rlim_cur=8192*1024,rlim_max=RLIM64_INFINITY})
= 0
    14160 munmap(0x718f19aba000,59259)          = 0
    14160 getRandom("\xe7\x98\x27\x86\x1b\x12\x92\x1b",8,GRND_NONBLOCK)
= 8
    14160 brk(NULL)                             = 0x5c955baf9000
    14160 brk(0x5c955bb1a000)                   = 0x5c955bb1a000
    14160 openat(AT_FDCWD,"1.txt",O_WRONLY|O_CREAT|O_TRUNC,0666) = 3
    14160 fstat(0,{st_mode=S_IFIFO|0600,st_size=0,...}) = 0
    14160 read(0, <unfinished ...>
    14159 <... read resumed>"Hello world\n",1024) = 12
    14159 write(4,"Hello world\n",12)           = 12
    14160 <... read resumed>"Hello world\n",4096) = 12
    14159 read(0,"invalid line\n",1024)         = 13
    14159 write(4,"invalid line\n",13 <unfinished ...>
    14160 fstat(3, <unfinished ...>
    14159 <... write resumed>)                   = 13
    14159 read(0,"Another valid line\n",1024) = 19
    14159 write(4,"Another valid line\n",19 <unfinished ...>
    14160 <... fstat resumed>{st_mode=S_IFREG|0664,st_size=0,...}) = 0
    14159 <... write resumed>)                   = 19
    14159 read(0, <unfinished ...>
    14160 read(0, <unfinished ...>
    14159 <... read resumed>"test string\n",1024) = 12
    14159 write(4,"test string\n",12)           = 12
    14160 <... read resumed>"invalid line\nAnother valid
line\n"... ,4096) = 44
    14159 read(0,"Last example\n",1024)         = 13
    14159 write(4,"Last example\n",13)          = 13
    14160 fstat(1, <unfinished ...>
    14159 read(0,"\n",1024)                     = 1
    14159 write(4,"\n",1 <unfinished ...>
    14160 <... fstat resumed>{st_mode=S_IFIFO|0600,st_size=0,...}) = 0
    14159 <... write resumed>)                   = 1
    14159 read(0, <unfinished ...>
    14160 read(0,"Last example\n\n",4096) = 14
    14160 read(0, <unfinished ...>
    14159 <... read resumed>"\n",1024)          = 1
    14159 write(4,"\n",1)                       = 1
    14160 <... read resumed>"\n",4096)          = 1
    14159 read(0, <unfinished ...>
    14160 read(0, <unfinished ...>
    14159 <... read resumed>"",1024)             = 0
    14159 close(4)                             = 0
    14160 <... read resumed>"",4096)            = 0
    14159 read(5, <unfinished ...>
    14160 write(3,"Hello world\nAnother valid line\nL"... ,44) = 44
    14160 close(3)                             = 0

```

```

14160 write(1,"line invalid line\n is not valid\n"... ,103) = 103
14159 <... read resumed>"line invalid line\n is not
valid\n"... ,256) = 103
14160 exit_group(0) = ?
14159 write(1,"line invalid line\n is not valid\n"... ,103) = 103
14160 +++ exited with 0 +++
14159 ---SIGCHLD
{si_signo=SIGCHLD,si_code=CLD_EXITED,si_pid=14160,si_uid=1000,si_status=0,si_utime=0,
---
14159 read(5,"",256) = 0
14159 close(5) = 0
14159 wait4(-1,NULL,0,NULL) = 14160
14159 exit_group(0) = ?
14159 +++ exited with 0 +++

```

Листинг 5: \*Strace логи\*